

UNIT III – Java New Features (With Definitions + Examples)

◆ 1. Functional Interface

A Functional Interface in Java is an interface that contains exactly one abstract method, making it suitable for lambda expressions and functional programming. It may include multiple default and static methods but must have only one abstract method. These interfaces enable passing behavior as parameters and are widely used in Java Streams and APIs like Runnable and Comparator.

✓ Example:

```
@FunctionalInterface
interface MyInterface {
    void show();
}

class Test {
    public static void main(String[] args) {
        MyInterface obj = () -> System.out.println("Hello Functional Interface");
        obj.show();
    }
}
```

◆ 2. Lambda Expression

A Lambda Expression is a concise way to represent an anonymous function that can be passed as an argument. It eliminates the need for verbose anonymous classes and improves readability. Introduced in Java 8, it supports functional programming and allows operations like filtering and sorting collections efficiently using minimal syntax.

✓ Example:

```
interface Add {
    int sum(int a, int b);
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Add obj = (a, b) -> a + b;  
        System.out.println(obj.sum(5, 10));  
    }  
}
```

◆ 3. Method References

Method References provide a shorthand notation for lambda expressions by referring to an existing method. Instead of writing a full lambda expression, developers can directly use a method name. This improves readability and reduces code duplication. It supports referencing static methods, instance methods, and constructors.

✓ Example:

```
import java.util.*;  
  
public class Main {  
    public static void main(String[] args) {  
        List<String> list = Arrays.asList("A", "B", "C");  
        list.forEach(System.out::println);  
    }  
}
```

◆ 4. Stream API

The Stream API is a powerful feature introduced in Java 8 that allows processing of collections in a functional style. It supports operations like filtering, mapping, and reducing data without modifying the original collection. Streams can be sequential or parallel, improving performance and enabling cleaner, more expressive code compared to traditional loops.

✓ Example:

```
import java.util.*;  
  
public class Main {  
    public static void main(String[] args) {  
        List<Integer> list = Arrays.asList(5, 10, 15, 20);  
  
        list.stream()
```

```
.filter(x -> x > 10)
.forEach(System.out::println);
}
}
```

◆ 5. Default Methods

Default Methods are methods inside interfaces with a defined body using the default keyword. They were introduced to allow adding new functionality to interfaces without breaking existing implementations. Classes implementing the interface inherit these methods automatically and can override them if needed. This feature ensures backward compatibility .

✓ Example:

```
interface Vehicle {
    default void horn() {
        System.out.println("Beep Beep");
    }
}

class Car implements Vehicle {
    public static void main(String[] args) {
        Car c = new Car();
        c.horn();
    }
}
```

◆ 6. Static Methods in Interface

Static Methods in interfaces belong to the interface itself and are not inherited by implementing classes. They are used for utility functions related to the interface. These methods must be called using the interface name, which helps in better code organization and avoids unnecessary class creation .

✓ Example:

```
interface MathUtil {
    static int add(int a, int b) {
        return a + b;
    }
}
```

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println(MathUtil.add(5, 10));  
    }  
}
```

◆ 7. Base64 Encoding and Decoding

Base64 encoding is used to convert binary data into text format so that it can be safely transmitted over networks or stored in text-based formats. Java provides built-in classes for encoding and decoding. It is commonly used in APIs, email attachments, and security mechanisms like token encoding.

✓ Example:

```
import java.util.Base64;  
  
public class Main {  
    public static void main(String[] args) {  
        String str = "Hello";  
        String encoded = Base64.getEncoder().encodeToString(str.getBytes());  
        String decoded = new String(Base64.getDecoder().decode(encoded));  
  
        System.out.println(encoded);  
        System.out.println(decoded);  
    }  
}
```

◆ 8. forEach Method

The `forEach` method is a default method available in the `Iterable` interface that allows iteration over a collection using lambda expressions. It simplifies looping and improves readability compared to traditional for-loops. It is often used with streams and functional programming constructs.

✓ Example:

```
import java.util.*;  
  
public class Main {  
    public static void main(String[] args) {  
        List<String> list = Arrays.asList("Java", "Python", "C++");
```

```
list.forEach(x -> System.out.println(x));  
}  
}
```

◆ 9. Try-With-Resources

Try-With-Resources is a feature that automatically closes resources like files or database connections after execution. It ensures that resources are properly managed and prevents memory leaks. It simplifies code by removing the need for finally blocks and is used with classes implementing `AutoCloseable`.

✓ Example:

```
import java.io.*;  
  
public class Main {  
    public static void main(String[] args) {  
        try (FileReader fr = new FileReader("file.txt")) {  
            System.out.println("File opened successfully");  
        } catch (Exception e) {  
            System.out.println(e);  
        }  
    }  
}
```

◆ 10. Type Annotations

Type Annotations allow annotations to be applied to any type usage such as variables, generics, or return types. They improve code safety by enabling tools to detect errors like null pointer exceptions. Introduced in Java 8, they are mainly used in frameworks and static analysis tools.

✓ Example:

```
List<@NonNull String> list;
```

◆ 11. Repeating Annotations

Repeating Annotations allow the same annotation to be applied multiple times to a single element. This feature simplifies code and improves readability when multiple annotations of the same type are required. It is implemented using the `@Repeatable` annotation.

✓ Example:

```
@Repeatable(Subjects.class)
@interface Subject {
    String value();
}

@Subject("Math")
@Subject("Science")
class Student {}
```

◆ 12. Java Module System

The Java Module System (introduced in Java 9) helps in organizing code into modules. It improves maintainability, security, and performance by allowing developers to define dependencies explicitly. Modules make large applications easier to manage and reduce unnecessary access to internal APIs.

✓ Example:

```
module mymodule {
    requires java.base;
}
```

◆ 13. Diamond Operator with Anonymous Class

The Diamond Operator (<>) simplifies generic type declarations by allowing the compiler to infer types automatically. It reduces code verbosity and improves readability. It can also be used with anonymous classes to avoid repetition of type parameters.

✓ Example:

```
List<String> list = new ArrayList<>();
```

◆ 14. Local Variable Type Inference (var)

The var keyword allows the compiler to infer the type of local variables automatically. It reduces verbosity and improves readability but should be used carefully to maintain code clarity. It can only be used for local variables.

✓ Example:

```
var x = 10;  
var name = "Java";
```

◆ 15. Switch Expressions

Switch Expressions allow returning values directly from switch statements, making code more concise and less error-prone. Introduced in Java 12, they eliminate the need for break statements and support arrow syntax.

✓ Example:

```
int result = switch (day) {  
    case 1 -> 10;  
    default -> 0;  
};
```

◆ 16. Yield Keyword

The yield keyword is used in switch expressions to return a value from a block. It replaces traditional return statements inside switch blocks and improves readability.

✓ Example:

```
int result = switch(day) {  
    case 1: yield 10;  
    default: yield 0;  
};
```

◆ 17. Text Blocks

Text Blocks allow writing multi-line strings easily using triple quotes. They improve readability and remove the need for escape characters.

✓ Example:

```
String text = """"  
Hello  
Java  
World
```

.....
,

◆ 18. Records

Records are special classes used to store immutable data. They automatically generate constructors, getters, equals(), hashCode(), and toString() methods, reducing boilerplate code.

✓ Example:

```
record Student(int id, String name) {}
```

◆ 19. Sealed Classes

Sealed Classes restrict which classes can extend or implement them. This improves security and control over inheritance.

✓ Example:

```
sealed class Shape permits Circle, Square {}
```

◆ THREAD LIFE CYCLE (IMPORTANT ADDITION)

◆ Definition

Thread Life Cycle in Java represents the different states a thread goes through during its execution. It helps in understanding thread behavior and managing multithreading efficiently. The lifecycle includes states like New, Runnable, Running, Blocked, Waiting, and Terminated. Proper understanding ensures better thread synchronization and performance.

◆ States of Thread

1. **New** → Thread created
2. **Runnable** → Ready to run
3. **Running** → Executing

4. **Blocked/Waiting** → Waiting for resource
 5. **Terminated** → Finished
-

◆ Example:

```
class MyThread extends Thread {  
    public void run() {  
        System.out.println("Thread Running");  
    }  
  
    public static void main(String[] args) {  
        MyThread t = new MyThread();  
        t.start();  
    }  
}
```

◆ UNIT IV – Collection Framework (Detailed + Example)

👉 Already aligned with your file content

◆ Collection in Java

A Collection in Java is a framework that provides architecture to store and manipulate a group of objects dynamically. It includes interfaces, classes, and algorithms to handle data efficiently. Collections are part of the `java.util` package and support operations like insertion, deletion, and searching, making data handling flexible and efficient .

◆ Iterator Interface

Iterator is used to traverse elements in a collection one by one. It provides methods like `hasNext()` and `next()` for iteration. It is useful for safe removal of elements during traversal.

✅ Example:

```
Iterator<Integer> it = list.iterator();
while(it.hasNext()) {
    System.out.println(it.next());
}
```

◆ List Interface

The List interface represents an ordered collection that allows duplicate elements. It maintains insertion order and provides index-based access to elements. It is widely used for storing data where order matters.

✓ Example:

```
List<String> list = new ArrayList<>();
list.add("Java");
```

◆ ArrayList

ArrayList is a dynamic array implementation of the List interface that allows storing elements in a resizable structure. Unlike normal arrays, it can grow or shrink automatically as elements are added or removed. It provides fast random access using index positions because it internally uses an array. However, insertion and deletion in the middle are slower due to shifting of elements. It is not synchronized, making it faster but not thread-safe.

✓ Example:

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        ArrayList<String> list = new ArrayList<>();
        list.add("Java");
        list.add("Python");
        list.add("C++");

        System.out.println(list.get(1)); // Python
    }
}
```

◆ LinkedList

LinkedList is a class that implements the List and Queue interfaces using a doubly linked list data structure. Each element (node) contains data and references to the previous and next nodes. It allows efficient insertion and deletion operations because elements do not need to be shifted. However, accessing elements by index is slower compared to ArrayList because traversal is required. It is suitable when frequent modifications are needed.

✓ Example:

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        LinkedList<Integer> list = new LinkedList<>();
        list.add(10);
        list.add(20);
        list.addFirst(5);

        System.out.println(list);
    }
}
```

◆ Vector

Vector is a legacy class that implements the List interface and is similar to ArrayList but synchronized. This means it is thread-safe and can be used in multi-threaded environments without additional synchronization. However, due to synchronization overhead, it is slower than ArrayList. It is rarely used in modern applications, where ArrayList or Collections.synchronizedList() is preferred.

✓ Example:

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Vector<Integer> v = new Vector<>();
        v.add(10);
        v.add(20);

        System.out.println(v);
    }
}
```

```
}
```

◆ Stack

Stack is a subclass of Vector that represents a Last In First Out (LIFO) data structure. It allows operations like push (insert), pop (remove), and peek (view top element). The last element added is the first one to be removed. It is commonly used in recursion, expression evaluation, and backtracking algorithms.

✓ Example:

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Stack<Integer> s = new Stack<>();
        s.push(10);
        s.push(20);

        System.out.println(s.pop()); // 20
    }
}
```

◆ Queue Interface

Queue is an interface that represents a First In First Out (FIFO) data structure. Elements are added at the rear and removed from the front. It is commonly used in scheduling, buffering, and task processing systems. Java provides implementations like PriorityQueue and LinkedList. Some queues may prioritize elements instead of strict FIFO order.

✓ Example:

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Queue<Integer> q = new LinkedList<>();
        q.add(10);
        q.add(20);

        System.out.println(q.poll()); // 10
    }
}
```

```
}
```

◆ Set Interface

Set is a collection interface that does not allow duplicate elements. It models a mathematical set and is used when uniqueness of elements is required. It does not guarantee order (except `LinkedHashSet`) and provides faster search operations. Common implementations include `HashSet`, `LinkedHashSet`, and `TreeSet`. It is widely used in applications like removing duplicates and membership testing.

✓ Example:

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Set<Integer> set = new HashSet<>();
        set.add(10);
        set.add(10); // duplicate ignored
        set.add(20);

        System.out.println(set);
    }
}
```

◆ Map Interface

Map is not a part of the Collection interface but is part of the Collection Framework. It stores data in key-value pairs, where each key is unique and maps to a specific value. It is useful for fast data retrieval based on keys. Common implementations include `HashMap`, `LinkedHashMap`, `TreeMap`, and `Hashtable`. It is widely used in databases, caching, and configuration storage.

✓ Example:

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Map<Integer, String> map = new HashMap<>();
        map.put(1, "Java");
        map.put(2, "Python");
    }
}
```

```
        System.out.println(map.get(1)); // Java
    }
}
```

◆ Comparable Interface

Comparable is an interface used to define the natural sorting order of objects. A class implements Comparable and overrides the `compareTo()` method to specify sorting logic. It is used when only one type of sorting is required. It is commonly used in sorting collections like ArrayList using `Collections.sort()`.

✓ Example:

```
class Student implements Comparable<Student> {
    int marks;

    Student(int marks) {
        this.marks = marks;
    }

    public int compareTo(Student s) {
        return this.marks - s.marks;
    }
}
```

◆ Comparator Interface

Comparator is an interface used to define custom sorting logic externally, allowing multiple sorting strategies. It overrides the `compare()` method and is used when different sorting criteria are required. It is more flexible than Comparable and widely used with `Collections.sort()` and streams.

✓ Example:

```
import java.util.*;

class Main {
    public static void main(String[] args) {
        List<Integer> list = Arrays.asList(5, 2, 8);

        Collections.sort(list, (a, b) -> b - a); // descending
    }
}
```

```
        System.out.println(list);  
    }  
}
```